สร้าง RESTful API ด้วยภาษา TypeScript / JavaScript

ปูพื้นฐานฉบับรวบรัด
เหมาะสำหรับคนรีบกลับบ้าน

Why?

เรียนทำไม?

Why we care about **Web protocols**?

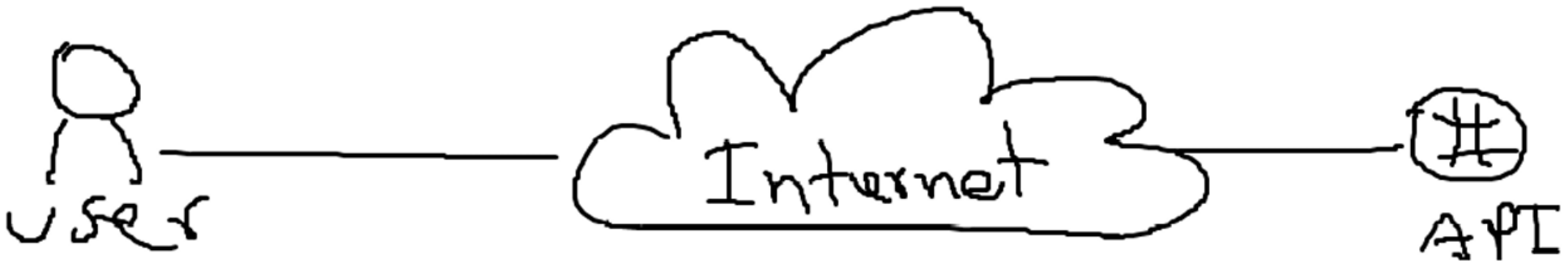
- ทุกคนใช้ อะไรก็ใช้ IoT ก็ใช้ AI ก็ใช้ เป็นมาตรฐานเดียวกัน
- เชื่อม IoT, Server, Web App และ Mobile App เข้าด้วยกันได้ทั้งหมด
- ส่งข้อมูลผ่านเครือข่ายและอินเทอร์เน็ตได้ง่าย
- Light Weight but Secure
- เรียนรู้ไว้ประดับหัว

RESTful API

คือ?

What’s REST and RESTful

- REST คือ “สถาปัตยกรรม”
- RESTful คือบริการผ่านเว็บที่ใช้สถาปัตยกรรมแบบ “REST”



การออกแบบ RESTful API

- เรียกใช้งานผ่าน Protocol HTTP รวมถึง HTTPs ด้วย
  - ดูสถานะการทำงานด้วย HTTP Response Code
  - การกระทำต่าง ๆ (CRUD) อ้างอิงจาก HTTP Method
- ควรเป็น Stateless ไม่มีการจดจำ Session
- ควรเป็น Resource Based
- ตอบกลับข้อมูลในรูปแบบของ JSON, XML หรือ อื่น ๆ
- **Client-Server Based** การทำงานของทั้งผู้เรียก (Client) หรือผู้ให้บริการ (Server) ไม่จำเป็นต้อง “ยุ่ง” หรือรู้เรื่องของกันและกัน
- และอื่น ๆ

การออกแบบ RESTful API URL

https://www.services.com/api/v1/books/

- Version ของ API ที่กำลังเรียกใช้
- Resource ที่เราต้องการ



การออกแบบ RESTful API URL



https://www.services.com/api/v1/books/1

- ID ของหนังสือที่ต้องการ
- Version ของ API ที่กำลังเรียกใช้
- Resource ที่เราต้องการ

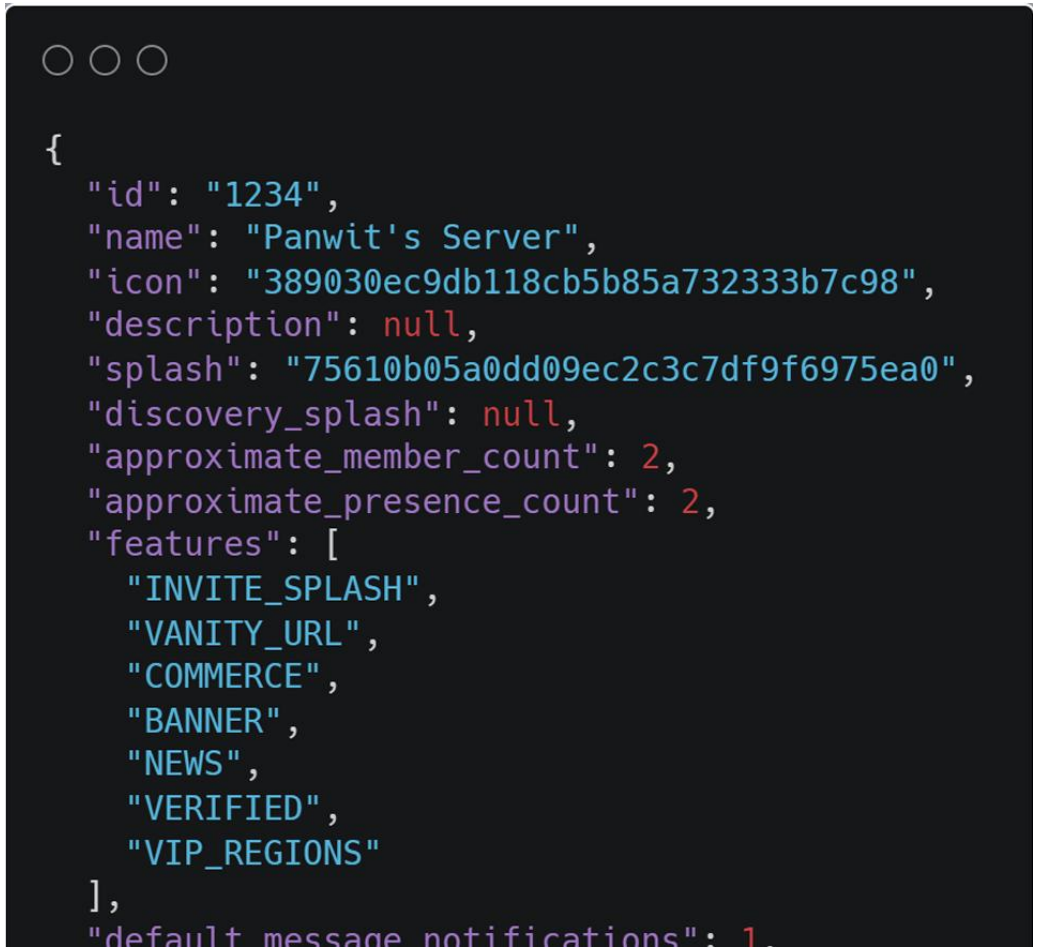
การออกแบบ RESTful Versioning

https://notdiscord.com/api/v9/guilds/1234

https://notdiscord.com/api/v10/guilds/1234

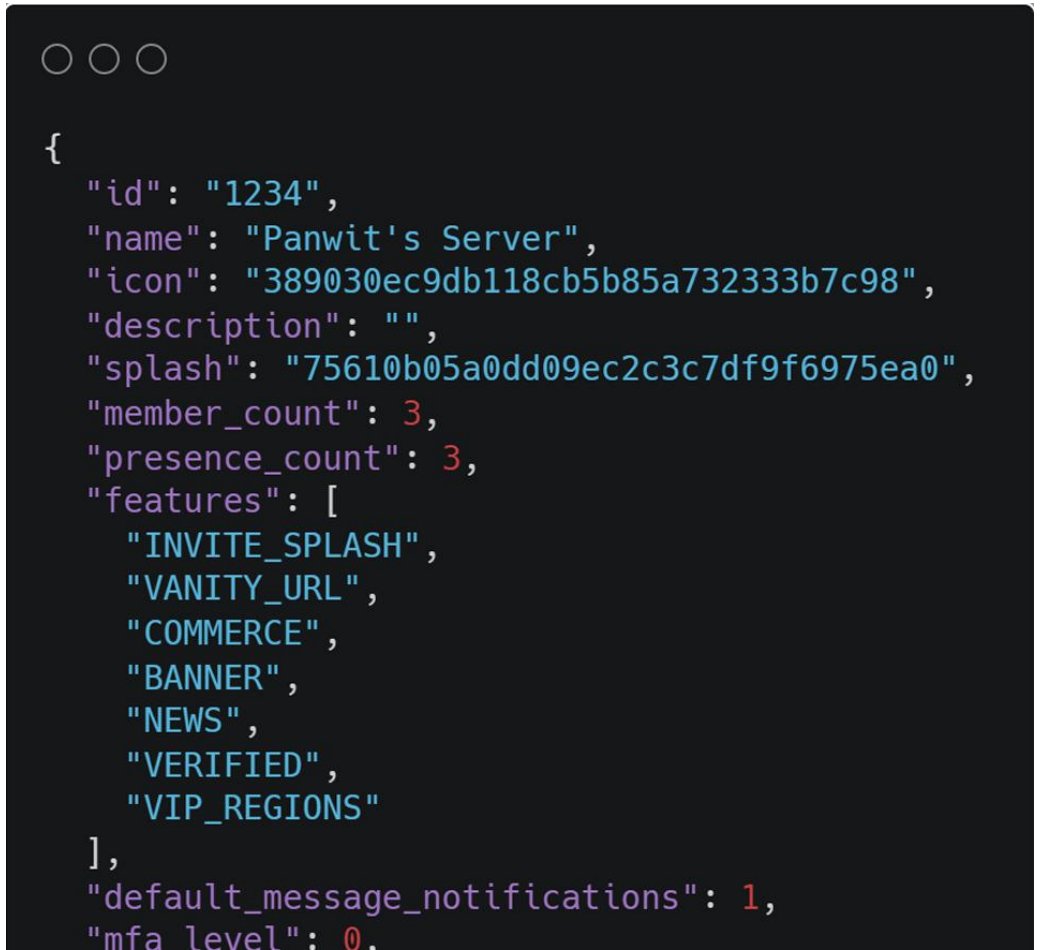
การออกแบบ RESTful Versioning

https://notdiscord.com/api/v9/guilds/1234



```
{
  "id": "1234",
  "name": "Panwit's Server",
  "icon": "389030ec9db118cb5b85a732333b7c98",
  "description": null,
  "splash": "75610b05a0dd09ec2c3c7df9f6975ea0",
  "discovery_splash": null,
  "approximate_member_count": 2,
  "approximate_presence_count": 2,
  "features": [
    "INVITE_SPLASH",
    "VANITY_URL",
    "COMMERCE",
    "BANNER",
    "NEWS",
    "VERIFIED",
    "VIP_REGIONS"
  ],
  "default_message_notifications": 1,
```

https://notdiscord.com/api/v10/guilds/1234



```
{
  "id": "1234",
  "name": "Panwit's Server",
  "icon": "389030ec9db118cb5b85a732333b7c98",
  "description": "",
  "splash": "75610b05a0dd09ec2c3c7df9f6975ea0",
  "member_count": 3,
  "presence_count": 3,
  "features": [
    "INVITE_SPLASH",
    "VANITY_URL",
    "COMMERCE",
    "BANNER",
    "NEWS",
    "VERIFIED",
    "VIP_REGIONS"
  ],
  "default_message_notifications": 1,
  "mfa_level": 0,
```

การออกแบบ RESTful Versioning

https://notdiscord.com/api/v9/guilds/1234

```
{
  "id": "1234",
  "name": "Panwit's Server",
  "icon": "389030ec9db118cb5b85a732333b7c98",
  "description": null,
  "splash": "75610b05a0dd09ec2c3c7df9f6975ea0",
  "discovery_splash": null,
  "approximate_member_count": 2,
  "approximate_presence_count": 2,
  "features": [
    "INVITE_SPLASH",
    "VANITY_URL",
    "COMMERCE",
    "BANNER",
    "NEWS",
    "VERIFIED",
    "VIP_REGIONS"
  ],
  "default_message_notifications": 1,
```

https://notdiscord.com/api/v10/guilds/1234

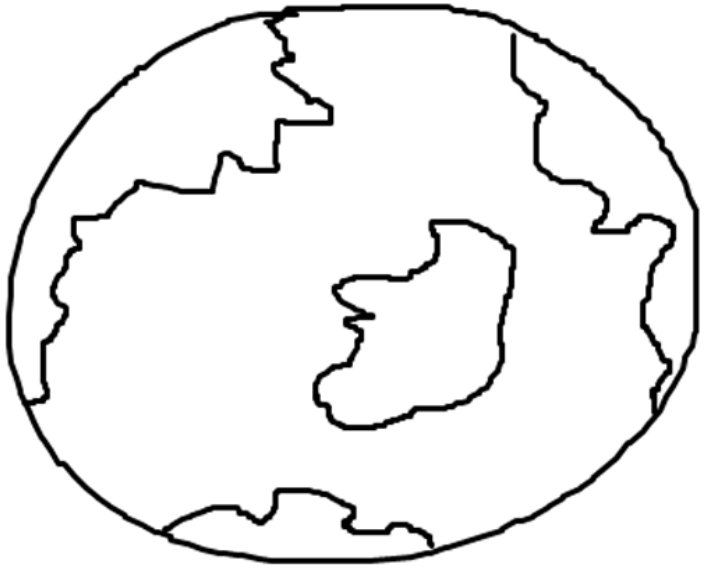
```
{
  "id": "1234",
  "name": "Panwit's Server",
  "icon": "389030ec9db118cb5b85a732333b7c98",
  "description": "",
  "splash": "75610b05a0dd09ec2c3c7df9f6975ea0",
  "member_count": 3,
  "presence_count": 3,
  "features": [
    "INVITE_SPLASH",
    "VANITY_URL",
    "COMMERCE",
    "BANNER",
    "NEWS",
    "VERIFIED",
    "VIP_REGIONS"
  ],
  "default_message_notifications": 1,
  "mfa_level": 0,
```

Hono

Oh, no! it’s typescript

What’s Hono?

- **Hono** is a unopinionated modern web framework for building web applications and APIs with **TypeScript** or **JavaScript**.
- Inspired by frameworks like Express. but optimized for **speed** and **simplicity**.
- Support multiple runtimes.
- Packed with adapters, tools and examples.



Hono Example Code (ตัวอย่างที่ 0)

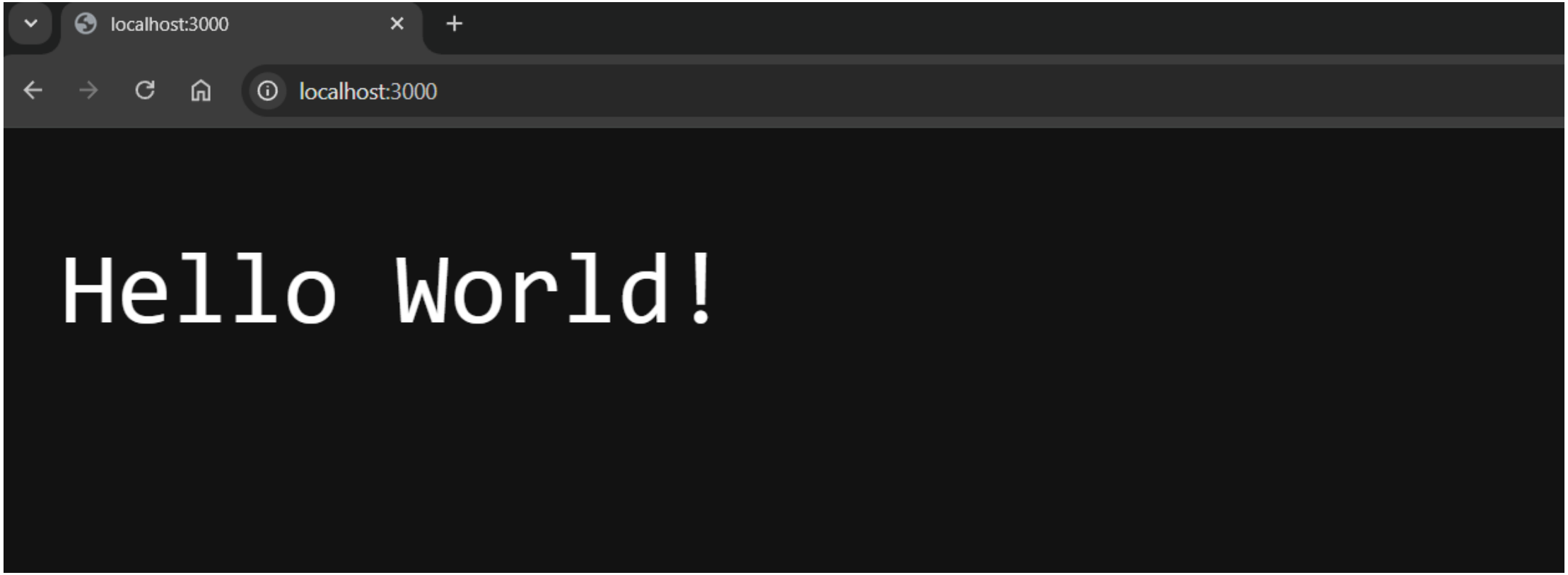
```
import { serve } from "@hono/node-server";
import { Hono } from "hono";

const app = new Hono();

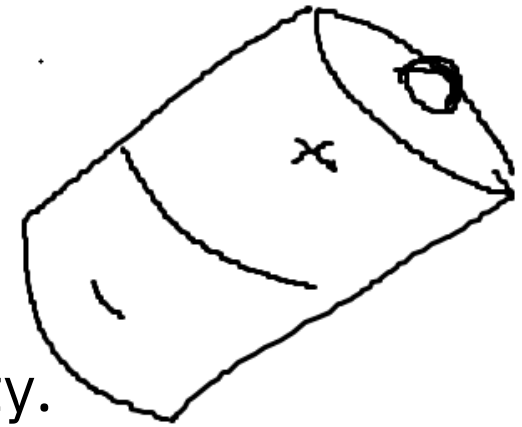
app.get("/", (c) => {
  return c.text("Hello World!");
});

serve(
  {
    fetch: app.fetch,
    port: 3000,
  },
  (info) => {
    console.log(`Server is running on
http://localhost:${info.port}`);
  }
);
```

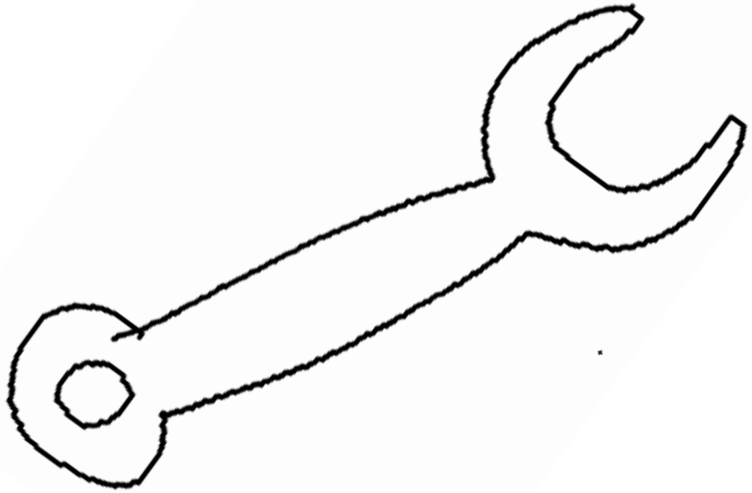
Example Response



Hono Alternatives



- Alternatives for Hono for example
  - Elysia.js for more developer friendly and typesafety.
  - Express.js for plainer and larger communities, but old and not maintained well.
  - Nest.js for more opinionated tools and battery included.
  - Adonis.js is more like Laravel using MVC design and battery included.



Lab Pre-Requisites

- Node.js 24 LTS (or something similar)
  - Windows can download directly from nodejs.org
  - MacOS
    - Use Homebrew “brew install nvm” and “nvm install 24”
    - Use NVM install script → Search “nvm-sh/nvm” on Google and pick Github
  - Linux
    - Use NVM, same same as second MacOS option. It’s compatible.
- Database Manager (Pick One)
  - DBeaver ← Recommended
  - HeidiSQL
  - Navicat
- Code Editor / IDE → Visual Studio Code
- Vercel Account

Example Lab Code



github.com/n0uur/iot-lab-2026

ตัวอย่างที่ 1

Hono Example

github.com/n0uur/iot-lab-2025

Database

We need to “Store” our data.

We will store them in **Database.**

PostgreSQL

ตัวอย่างที่ 2.1

อ่าน เขียนข้อมูล บนฐานข้อมูล SQLite (Local)

How to DEPLOY my application.

to the internet.

We will use Railway or Vercel for Deployment.

What’s Vercel?



- Cloud Platform Provider ให้บริการ Static Website Hosting และ Serverless Function รวมถึงบริการอื่น ๆ เช่น ฐานข้อมูล พื้นที่เก็บข้อมูล
  - ข้อดีมี **Free Tier**
    - มีบริการครบจบ
    - มีระบบ CI/CD ออโต้ ไม่ต้องทำไรเยอะ Setup น้อย ใช้ง่าย
  - ข้อเสีย
    - ถ้าใช้เยอะ จะเสียเงินแพงกว่าบริการแบบดั้งเดิม (เช่า Server + Setup เอง)
    - รองรับ Node.js เป็น Native เราใช้ Python ต้องเล่นท่านิดหน่อย ถึงใช้งานได้
  - ข้อควรระวัง
    - อย่าใช้เยอะ เดี๋ยวจะเจ็บตัว
    - บริการส่วนใหญ่ Vercel ก็เอาบริการที่อื่นมาย้อมแมวขายเรา เช่น Cloudflare, AWS, Neon แล้วบวก ราคาเพิ่ม ดังนั้นอย่าคิดเสียเงินให้ Vercel ไม่คุ้ม
    - ใช้แล้ว บางบริการ ย้ายออกไปใช้เจ้าอื่นยาก, โดน Lock-In

Setup “Vercel”

สร้างบัญชี และฐานข้อมูลสำหรับใช้งาน

ตัออย่างที่ 2.2

อ่านเขียนข้อมูลจากฐานข้อมูล PostgreSQL ที่สร้างบน ~~Railway~~ Vercel

หัวข้อเพิ่มเติมสำหรับศึกษาด้วยตนเอง

- Error Handling
- CORS Cross-Origin Resource Sharing
- User Authentications
- Database Migration